

Owmee System Architecture and Design

Version: 2026-06-29

1. Executive Summary

Owmee is a trust-first C2C resale platform for India. The product is not a chat marketplace and not a meetup marketplace. Owmee owns the transaction journey: AI-assisted listing creation, structured offer/payment capture, seller readiness, FE pickup inspection, hub/courier routing, delivery proof, buyer acceptance/dispute windows, and seller payout controls.

The current architecture is a modular monolith with clear provider boundaries. The backend is FastAPI with Postgres as the source of truth, Redis for OTP, queues, and rate limiting, R2 for media, React Native for the buyer/seller/FE mobile app, and a Vite/React admin console for operations.

2. Architecture Principles

- 1 Postgres is the system of record.
- 2 Business modules own domain rules. Routers should stay thin.
- 3 Provider integrations go through adapters, never direct SDK calls from business logic.
- 4 No vendor calls from high-traffic read paths such as home feed, search, or listing detail.
- 5 Immutable transaction snapshots are created before logistics starts.
- 6 Buyer and seller never chat. All communication is structured.
- 7 Payment intent is not an order. Captured payment is the order trigger.
- 8 Payment capture does not release seller payout.
- 9 FE inspection is mandatory before Owmee trusts item condition.
- 10 Admin has manual routing authority during pilot and launch.

3. High-Level System Map

Mobile app

- Buyer, seller, FE screens
 - OTP login and address gate
 - AI listing camera and review
 - Browse, wishlist, offer, checkout
 - Seller inventory and readiness
 - FE onboarding, pickup, delivery, direct acquisition

Admin web

- Ops, finance, risk, warehouse, community, FE management
 - Moderation queues
 - Pickup, hub, return, refund queues
 - FE onboarding and certification
 - Direct acquisition finance and warehouse controls

Backend API

- FastAPI modular monolith
 - Identity/auth, verification, KYC
 - Listings, AI drafts, media
 - Offers, payments, transactions
 - Logistics, FE, direct acquisition
 - Admin, analytics, compliance, notifications

Worker

- Background AI/media/queue processing
 - R2 image handling
 - Gemini draft analysis
 - Hero image cleanup
 - Bounded retries and metrics

Infrastructure

Postgres, Redis, R2, Render, Sentry
Providers: Gemini, MSG91, Razorpay, Bureau, KYC partner, geo, FCM

4. Runtime Components

Component	Path	Responsibility
Mobile app	mobile/src	Buyer, seller, FE user experience
Admin app	admin/src	Operations console for launch teams
API	backend/app/main.py	FastAPI application and route composition
Worker	backend/app/workers/main.py	Queue and background processing
DB migrations	backend/app/db/migrations	Schema evolution
Tests	backend/tests, mobile scripts	Contract and regression coverage

5. Backend Module Ownership

Module	Primary ownership
identity_auth	OTP login, sessions, devices, profile, user addresses, location
verification	Fraud/risk checks, action policy, trust summary, admin risk queue
kyc	Aadhaar/PAN/liveness/payout checks and consent events
ai_assistant	Listing draft analysis, prompts, Gemini adapter, price suggestions
media	R2 media operations, hero cleanup, image provider abstraction
listings	Categories, listing CRUD, publish validation, seller inventory
offers	Offers, buy-now, payment attempts, readiness, notifications
payments	Payment aggregator adapter and Razorpay webhook
transactions	Logistics state, shipment, refund, return, payout, snapshots
field_executive	FE profiles, onboarding, visits, earnings, issue/NPS capture
direct_acquisition	Owmee Direct seller-to-Owmee acquisition flow
admin	Admin auth, listing/KYC queues, audit log, reports/disputes
community	Community referral and verification
analytics	Client/admin event tracking
compliance	Data export and erasure requests
notifications	Notification events, preferences, push adapter
stuck_workflow	Ops alerting for stuck states

6. Key Data Model Groups

Domain	Tables
Identity	users, user_addresses, sessions, devices, auth_events, phone_change_requests
Trust	verification_checks, risk_decisions, kyc_verifications, kyc_events, consent_events
Listings	categories, listings, listing_images, listing_snapshots
Commerce	offers, reservations, transactions, payment_links, ratings, wishlists
Notifications	notification_events, notification_preferences

Domain	Tables
FE	field_executives, fe_visits, fe_visit_issues, fe_visit_nps, fe_earnings
Direct acquisition	direct_acquisition_bookings, acquisition_items, price_override_approvals, seller_account_ledger_entries
Ops/admin	admin_users, admin_audit_log, super_admin_actions, user_reports, disputes, user_blocks
Transactions/accounting	tds_annual_ledger, reconciliation_runs
Analytics/community	analytics_events, communities, community_verifications

7. Public API Surface

The app mounts route groups under `backend/app/main.py`:

Area	Prefix examples
Auth and profile	<code>/v1/auth</code> , <code>/v1/users/me/addresses</code> , <code>/v1/users/me/location</code>
Verification/KYC	<code>/v1/verification</code> , <code>/v1/kyc</code>
Listings and AI drafts	<code>/v1/listings</code> , <code>/v1/listings/draft/*</code>
Feed/search	<code>/v1/feed/*</code> , <code>/v1/listings/search</code>
Offers/orders/payments	<code>/v1/offers</code> , <code>/v1/orders/buy-now</code> , <code>/v1/transactions/*</code> , <code>/v1/payments/webhook/razorpay</code>
Logistics	<code>/v1/admin/logistics/*</code> , <code>/v1/fe/pickups</code> , <code>/v1/fe/deliveries</code> , <code>/v1/transactions/{id}/tracking</code>
FE visits	<code>/v1/fe-visits/*</code> , <code>/v1/fe/visits/*</code> , <code>/v1/admin/fe-visits/*</code>
FE onboarding	<code>/v1/fe/onboarding/*</code> , <code>/v1/admin/fe-visits/fes/*</code>
Direct acquisition	<code>/v1/direct-sell/*</code> , <code>/v1/fe/bookings/*</code> , <code>/v1/ops/bookings/*</code> , <code>/v1/admin/listing-approvals/*</code>
Admin	<code>/v1/admin/*</code> , <code>/admin/*</code>
Compliance and analytics	<code>/v1/me/data-export</code> , <code>/v1/me/erase</code> , <code>/v1/analytics/*</code>

8. Mobile App Design

The mobile app is one React Native app with multiple role surfaces:

- Buyer: browse, search, wishlist, checkout, tracking, returns/disputes.
- Seller: AI listing creation, review/publish, inventory, offers, seller readiness, direct sell bookings.
- FE: onboarding, shift check-in, assigned visits, pickups, deliveries, direct acquisition QC.

Design-system primitives live under `mobile/src/components/ui` and tokens live under `mobile/src/utils/tokens.ts`. New work should prefer `Button`, `IconButton`, `BackButton`, `Card`, `Chip`, `StatusBadge`, `EmptyState`, and `ErrorState` over custom touchables, raw colors, or raw font sizes.

9. Admin App Design

The admin app is Vite/React. It supports:

- Analytics dashboards.
- Dispatch and stuck workflow queues.
- FE list and FE earnings.
- FE-assisted listings.

- Direct acquisition queue.
- Visit details.

The backend also has server-rendered admin pages under `/admin/*` for pilot operations such as pickups, hub, refunds, returns, disputes, and transaction detail. During launch, both admin surfaces can coexist, but ownership of each queue must stay explicit.

10. Provider Abstraction

Capability	Env switch	Adapter owner	Provider today
AI listing	AI_PROVIDER	ai_assistant.provider	Gemini
OTP SMS	SMS_PROVIDER	identity_auth.sms_adapter	MSG91
Fraud	FRAUD_PROVIDER	verification.fraud_adapter	Bureau/mock
Payment aggregator	PA_PROVIDER	payments.adapter	Razorpay/mock
KYC	KYC_PARTNER	kyc adapter layer	Digio/mock
Reverse geocoding	GEOCODING_PROVIDER	geo.provider	Photon
Push	PUSH_PROVIDER	notifications.push_adapter	FCM/noop

Provider boundaries are protected by tests in `backend/tests/test_provider_boundaries.py`. Unknown or empty production providers must fail fast instead of silently falling back to mocks.

11. AI Listing Architecture

The AI listing path is asynchronous for performance and memory safety:

```
Mobile requests upload slots
-> API creates listing_draft and R2 presigned URLs
-> Mobile uploads directly to R2
-> Mobile starts draft analysis
-> API enqueues Redis Stream job
-> Worker downloads one image at a time
-> Worker normalizes images and calls Gemini
-> Worker persists draft result and metrics
-> Mobile polls status
-> Seller reviews required checks
-> Backend validates and publishes listing
```

Important prompt and contract rules:

- The model must focus on the main product, not background noise.
- Titles must be specific; generic titles like "Other Pink" are banned.
- MRP can be used only when directly visible or responsibly evidenced.
- Category-specific requirements are deterministic in backend/mobile.
- Seller confirmation is required for missing or risky P0 fields.
- No extra Gemini call is added for category-family requirements.

12. Listing Review Contract

Launch category families:

- device
- toy
- book
- appliance
- other

P0 fields block publish when missing or unconfirmed:

- Photos.
- Category.
- Title.
- Price.
- Condition.
- Defect/missing-parts/safety disclosure.
- Pickup locality and fulfillment method.
- Category-specific attributes.
- Working/battery status where relevant.

P1 fields add trust but do not block:

- AI description.
- Brand unless category-critical.
- Material.
- Box/invoice.
- Purchase year.
- Extra notes/photos.
- Original price/MRP.

13. Commerce Architecture

Owmee supports structured offer and buy-now flows. There is no chat note.

Offer rules:

- One active offer per buyer/listing.
- Buyer can update price up to 3 times.
- Seller can accept, reject, or counter.
- Seller counter can be accepted or rejected; no re-counter.
- Rejected/expired offers can apply cooldown.

Payment rules:

- Payment attempts reserve inventory.
- Razorpay checkout can be in-app through the mobile SDK.
- Signed webhook or verified client confirmation advances capture.
- Invalid signature, underpayment, late capture, failure, cancel, or timeout must not reopen a cancelled sale incorrectly.
- Inventory is released on failed/cancelled/expired payment attempts.

14. Transaction And Logistics Architecture

Every paid transaction is Owmee-managed:

```
payment captured
-> seller readiness required
-> admin assigns pickup
-> FE pickup and inspection
-> hub custody
-> admin routes FE delivery or courier
-> delivery proof
-> buyer acceptance window
```

-> payout eligibility and release controls

The logistics state machine blocks skipping pickup or delivery. Admin pickup assignment requires seller readiness. Payout activities refuse release before pickup confirmation.

15. Direct Acquisition Architecture

Owmee Direct is seller-to-Owmee acquisition for controlled launch categories. It is separate from the C2C transaction flow.

seller books Owmee Direct pickup

- > ops assigns FE
- > FE arrival and seller OTP/QR verification
- > item-level photos and QC
- > price revision approval if needed
- > seller final OTP acceptance
- > FE requests payout-ready
- > finance posts seller ledger credit
- > warehouse receives custody
- > admin approves buyer-facing listing

Hard boundaries:

- FE cannot create offline bookings.
- FE cannot post payout.
- FE cannot mark warehouse inbound.
- Warehouse mismatch quarantines items.
- Buyer-facing listing requires warehouse inbound and admin approval.

16. Verification And Trust Architecture

Verification is a trust ladder:

- MSG91 proves phone control.
- Bureau provides fraud/risk signal.
- KYC proves legal identity, liveness, PAN, and payout account status.
- Owmee policy decides what the user can do.

KYC is not required for browsing, listing creation, normal offers, or buying. KYC is required for seller payout, verified seller trust boosts, and buyer refund/return/dispute escalation where legally attributable identity is needed.

17. Security, Privacy, And Compliance

Controls in the current design:

- No Aadhaar storage.
- No raw OTP in logs.
- No free-form buyer/seller chat evidence to preserve.
- Immutable address snapshots for logistics.
- Admin audit logs for sensitive actions.
- Provider-neutral verification and risk records.
- Data export and erasure endpoints.
- Sentry configured without default PII body capture.
- Production provider startup validation can fail closed.

18. Performance And Scalability

Current performance architecture:

- Async direct-to-R2 uploads for AI drafts.
- Redis Stream backed draft analysis jobs.
- One-photo-at-a-time worker processing.
- Low-media-resolution fast Gemini path with full fallback controls.
- Provider timing and token instrumentation.
- No vendor calls on read-heavy paths.
- Predeploy DB guardrails and Alembic head checks.
- Admin routing remains manual until pilot data is sufficient.

Startup infra priorities before public launch:

- Managed Postgres with backups and connection limits.
- Redis used for OTP, queues, rate limits, and revocation.
- Object storage with private originals and safe public display URLs.
- Sentry/error monitoring.
- Operational dashboards for stuck workflows and payment/logistics queues.

19. Quality Gates

Current high-signal checks:

- `backend/.venv/bin/python -m pytest tests -q`
- `npm run build` in admin
- `npm run tsc` in mobile
- `npm run lint:design` in mobile
- `npm run test:address-gate` in mobile
- `python -m app.db.predeploy` in backend
- `alembic heads` and `alembic current`
- Android/iOS native builds before release installation.

Known local warning:

- Python 3.14 can emit intermittent `asyncpg` cleanup warnings during test teardown. Production target is Python 3.12 and this has not been treated as a product failure.

20. Extraction Path

Keep the modular monolith until product-market fit and launch operations prove the separation points. Likely future extraction candidates:

- 1 AI/media worker service.
- 2 Payment/logistics orchestration service.
- 3 FE operations service.
- 4 Analytics/event warehouse.
- 5 Admin console as a separately deployed frontend.

Do not extract provider adapters prematurely. Stable domain contracts matter more than service boundaries during launch.